

SIMPLY FPU

by **Raymond Filiatreault**
Copyright 2003

Chap. 3 Instructions related to the FPU internals

The FPU instructions covered in this chapter perform no operation on numerical data. Their main purpose is to exchange information with the FPU or modify some of its settings. Several of them have both a "WAIT" and a "NO WAIT" variant (the latter having the letter "N" inserted immediately following the "F" in the mnemonic).

None of these instructions will raise an exception flag.

The described instructions are (in alphabetical order):

<u>FCLEX/FNCLEX</u>	CLeaR EXceptions
<u>FDECSTP</u>	DECrement STabl Pointer
<u>FFREE</u>	FREE a data register
<u>FINCSTP</u>	INCrement STabl Pointer
<u>FINIT/FNINIT</u>	INITialize the FPU
<u>FLDCW</u>	LoaD Control Word
<u>FLDENV</u>	LoaD ENVironment
<u>FRSTOR</u>	ReSTORe all registers
<u>FSAVE/FNSAVE</u>	SAVE state of FPU
<u>FSTCW/FNSTCW</u>	STore Control Word
<u>FSTENV/FNSTENV</u>	STore ENVironment
<u>FSTSW/FNSTSW</u>	STore Status Word
<u>FWAIT</u>	WAIT while FPU is busy

FWAIT (CPU waits while FPU is busy)

Syntax: **fwait** (no operand)

This instruction prevents the CPU from executing its next instruction if the Busy bit of the FPU's Status Word is set.

The FPU does not have direct access to the stream of instructions. It will start executing an instruction only when the CPU detects one in the code bits and transmits the information to the FPU. While the FPU is executing that instruction, the CPU can continue to execute in parallel other instructions which are not related to the FPU. Some of the FPU instructions being relatively slow, the CPU could execute several of its own instructions during that period.

The CPU also has some read/write access to the FPU's Status and Control registers even while the FPU is executing an instruction. In some cases, it may be desirable to read/write these registers without delay. But, in most cases, it is preferable or even necessary to wait until the FPU has completed the current instruction before proceeding with the read/write of these registers.

For example, a comparison by the FPU may require several clock cycles to execute and set the bits in the proper fields of the Status Word register. Waiting until the FPU has completed the operation and effectively updated the Status Word is a necessity if reading that Status Word is to have any meaning.

Many of the instructions which read or write to the FPU's Status and Control registers are automatically encoded with the fwait code.

Whenever the FPU is instructed to store a result (such as an integer) intended to be used by a CPU instruction, that CPU instruction should be preceded by an explicit FWAIT instruction. Although this may not be necessary on modern systems, the time cost is minimal and insures that the FPU has completed the task. (For example, storing the content of a data register to memory as an integer may take some 30 clock cycles to complete.)

```
fistp mem_variable
fwait
mov    eax,mem_variable
```

Note: Original versions of MASM were inserting an FWAIT instruction in front of every FPU instruction except those specified as "no-wait". This may have been necessary with the earlier co-processors but not with the more modern computers. Such coding may be observed if some old code is disassembled. The CPU can now poll the Busy bit of the Status Word whenever necessary before sending data processing instructions to the FPU.

FINIT / FNINIT (Initialize FPU)

Syntax: **finit** (no operand)
 fninit (no operand)

The FINIT and FNINIT have exactly the same coding except that the FINIT is automatically preceded by the FWAIT code.

This instruction initializes the FPU by resetting all the registers and flags to their default values. (Refer to [Chap.1](#) Description of FPU Internals for default values.)

Due to some of the restrictions of the FPU, it is good programming practice to take the precaution of initializing it before starting any computation, unless it is known that data has been left in it for further processing. If several sections of code use the FPU and the programmer is careless about not leaving the data registers "empty" in some of them, they can fill up very rapidly.

The "no-wait" variant may become necessary only if the programmer writes exception handling code and an endless wait is to be prevented.

Note: The FPU hardware is also used to process MMX/SSE instructions which may render its registers unusable for immediate FPU instructions. Initializing it would insure the clean-up for subsequent FPU instructions. Some of the API functions of Windows XP are also known to use MMX instructions without preserving the FPU registers and restoring them before leaving.

Using API functions under Windows XP while some of the FPU registers still contain valuable data must therefore be avoided.

FSTSW / FNSTSW (Store Status Word)

Syntax: **fstsw Dest**
 fnstsw Dest

The FSTSW and FNSTSW have exactly the same coding except that the FSTSW is automatically preceded by the FWAIT code.

This instruction stores the content of the [Status Word](#) register at a 16-bit WORD memory address (*Dest*). All valid addressing modes are acceptable for this memory destination. Specifying the size with indirect

addressing is not necessary because the WORD size is implicit in this instruction.

Ever since the 80287 co-processor, the destination can also be specified as the CPU's AX register.

This is the only FPU instruction which can transfer information directly from one of the FPU's registers to one of the CPU's general purpose registers.

The "no-wait" variant should only be used in exception handling code when it may be necessary to investigate which of the other flags of the Status Word may have been set. The "wait" variant should be used in most other cases. The remainder of this description will assume that the "wait" variant has been used.

The information in the Status Word is normally used to branch the program based on some result of the previous FPU instruction(s). A subsequent FWAIT instruction should always be included before the testing and/or branching code to insure that the information has effectively reached its specified destination.

If one or more of the exception flags need to be tested (such as for a division by zero and/or for an invalid operation), the related bits of the Status Word must then be tested individually (or in groups). Example code would be:

```

stword    dw    ?

fstsw stword    ;copy the status word to the memory variable stword
fwait          ;wait until the instruction is completed
test  stword,4+1 ;tests both the zero divide
                  ;and invalid exception flags together
jnz   error_code ;jump to the code section dealing with these exceptions
                  ;possibly testing further to define which one occurred
                  ;and issue the appropriate message or take other action

```

If the condition bits (C0-C3) need to be examined such as after a comparison, storing the Status Word in the AX register can be most useful. Those condition bits would then all be in the AH register which can be transferred directly into the CPU's flag register with the "sahf" instruction. With such a transfer,

the C0 bit becomes the CF (carry) flag,
 the C2 bit becomes the PF (parity) flag, and
 the C3 bit becomes the ZF (zero) flag.
 (The C1 bit is not associated with any flag.)

A simple conditional jump can then be used (instead of testing for set bits followed by a conditional jump). Example code would be:

```

fstsw ax        ;copy the status word to the AX register
fwait          ;wait until the instruction is completed
sahf           ;copy the condition bits in the CPU's flag register
ja   this_code ;jump to this code section if CF=C0=0 and ZF=C3=0

```

FSTCW / FNSTCW (Store Control Word)

Syntax: **fstcw Dest**
 fnstcw Dest

The FSTCW and FNSTCW have exactly the same coding except that the FSTCW is automatically preceded by the FWAIT code.

This instruction stores the content of the [Control Word](#) register at a 16-bit WORD memory address (*Dest*). All valid addressing modes are acceptable for this memory destination. Specifying the size with indirect addressing is not necessary because the WORD size is implicit in this instruction.

The "no-wait" variant could be used almost anytime because the content of the Control Word cannot be changed by any instruction except [FINIT/FNINIT](#) (and [FLDCW](#) which would be redundant immediately before an FSTCW instruction). Waiting for the FPU to finish executing its current instruction would thus not have any

effect on the information. The "wait" variant should however be used if it immediately follows an FINIT/FNINIT.

Although this Control Word contains some information about the settings of the FPU, the main purpose for getting a copy of it is generally for temporary storage. Some of the settings can then be selectively changed for a sequence of FPU operations, and the previous settings restored afterwards.

FLDCW (Load Control Word)

Syntax: **fldcw Src**

This instruction loads into the [Control Word](#) register the 16-bit WORD located at the specified memory address (*Src*). All valid addressing modes are acceptable for this memory source. Specifying the size with indirect addressing is not necessary because the WORD size is implicit in this instruction.

The purpose of this instruction is to modify the current settings of the FPU. The rounding control is probably the one which may need to be modified the most often. The following example code is based on the need to truncate a result before proceeding further, and the current Control Word may contain some other "rounding" mode.

```

oldcw    dw    ?

fstcw oldcw    ;get the current Control Word to retain all setting bits
            ;not related to the rounding control (RC) bits
fwait                    ;to insure the storage instruction is completed
mov     ax,oldcw
; and   ax,0F3FFh ;clears only the RC bits, leaving all other bits unchanged
            ;not necessary here because both bits will be set
or      ax,0C00h ;this will set both bits of the RC field to the truncating mode
            ;without affecting any of the other field's bits
push    eax      ;use the stack to store the modified Control Word in memory
fldcw   [esp]    ;load the modified Control Word

fxxxx                    ;other FPU instruction(s) needing the truncating mode

fldcw   oldcw    ;restore the previous Control Word
pop     eax      ;clean-up the stack
            ;this could also retrieve a 16-bit or 32-bit integer
            ;possibly returned by the "other FPU instruction(s)"

```

FCLEX / FNCLEX (Clear exceptions)

Syntax: **fclex** (no operand)
fnclex (no operand)

The FCLEX and FNCLEX have exactly the same coding except that the FCLEX is automatically preceded by the FWAIT code.

This instruction clears all the exception bits of the [Status Word](#).

The "no-wait" variant must be a part of any exception handling routine written by the programmer. Otherwise, if control is returned to the FPU without clearing the exceptions, more interrupts to handle the exception would be generated in an endless loop. (In such exception handling routines, the "wait" variant could generate an endless wait situation!)

When all exception handling is left to the FPU, it is still good programming practice to test occasionally the Status Word for some specific exceptions. If one (or more) has been detected which requires some branching in the code, exceptions should then be cleared to prevent future branching due to "old" exceptions.

FSAVE / FNSAVE (Save state of FPU)

Syntax: **fsave Dest**
 fnsave Dest

The FSAVE and FNSAVE have exactly the same coding except that the FSAVE is automatically preceded by the FWAIT code.

This instruction saves the entire state of the FPU at the specified address (*Dest*) and then initializes it as if an FINIT instruction had been executed. The required memory size is 108 bytes and the data will have the following format in 32-bit programming.

+0	WORD	Control Word
+2	WORD	(unused)
+4	WORD	Status Word
+6	WORD	(unused)
+8	WORD	Tag Word
+10	WORD	(unused)
+12	DWORD	Instruction Pointer
+16	WORD	Code Segment
+18	WORD	(unused)
+20	DWORD	Operand Address
+24	WORD	Data Segment
+26	WORD	(unused)
+28	TBYTE	ST(0)
+38	TBYTE	ST(1)
+48	TBYTE	ST(2)
+58	TBYTE	ST(3)
+68	TBYTE	ST(4)
+78	TBYTE	ST(5)
+88	TBYTE	ST(6)
+98	TBYTE	ST(7)

Fig.3.1 FPU Save State Format

Note: In 16-bit programming, the instruction pointer and operand address would have a WORD size. The first 28 bytes would thus be stored in only 14 bytes (7 WORDS without any unused memory) and followed immediately by the 8 TBYTE register data for a total of only 94 bytes.

The "no-wait" variant must be used if an exception handling routine written by the programmer needs the information returned by this instruction. Otherwise, the "wait" variant should be used.

This instruction would normally be used when the content of the FPU must be saved temporarily in order to use the FPU for some other reason. For example, a debugger program would be a typical application making use of this instruction if it needs to display the content of the FPU at a break point. It would then use the FPU to convert the saved floating point values and later restore the previous state of the FPU (with the [FRSTOR](#) instruction) so that the debugged program could continue to function normally.

This instruction should also be considered if a program must be executed under Windows XP and some of the API functions must be called while useful data is still in FPU registers. The registers can then be restored after the API call.

FRSTOR (Restore the state of the FPU)

Syntax: **frstor Src**

This instruction restores the state of the FPU previously saved in memory (by the [FSAVE](#) instruction) in the 108 bytes starting at the specified address (*Src*). The format of that data must be as depicted in [Fig.3.1](#) for 32-bit programming. (For 16-bit programming, see the [Note](#) under Fig.3.1).

Any alteration of that data in memory should generally be avoided. On rare occasions, the opportunity may be taken to modify the Control Word or clear exceptions before restoring the state of the FPU.

FSTENV / FNSTENV (Store environment)

Syntax: **fstenv** *Dest*
 fnstenv *Dest*

The FSTENV and FNSTENV have exactly the same coding except that the FSTENV is automatically preceded by the FWAIT code.

This instruction stores the environment state at the specified memory address (*Dest*). The required memory size is 28 bytes for 32-bit programming, having the same format as the first 28 bytes depicted in [Fig.3.1](#). (For 16-bit programming, see the [Note](#) under Fig.3.1).

The "no-wait" variant must be used if an exception handling routine written by the programmer needs some of the information returned by this instruction. Otherwise, the "wait" variant should be used to insure that the latest information will be made available.

Although the content of the Status Word and the Control Word can be retrieved by other means ([FSTSW](#) and [FSTCW](#) respectively), the Instruction Pointer and Operand Address of the last instruction processed by the FPU may be useful information when an exception is detected. Some code using this instruction could be inserted during the development phase to better identify potential programming flaws and lead to proper correction. (For example, a sub-routine could convert the data of the environment to alphanumeric and keep a log of each occurrence for later review and analysis).

FLDENV (Load environment)

Syntax: **fldenv** *Src*

This instruction loads the environment previously stored in memory (by the [FSTENV](#) instruction) in the 28 bytes starting at the specified address (*Src*). The format of that data must be as depicted in the first 28 bytes of [Fig.3.1](#) for 32-bit programming. (For 16-bit programming, see the [Note](#) under Fig.3.1).

The environment would normally have been stored following the detection of an exception. The opportunity could then be taken to modify the 16-bit registers for various reasons and then load the modified environment.

FFREE (Free a data register)

Syntax: **free** *st(x)*

This instruction modifies the [Tag Word](#) to indicate that the specified 80-bit data register is now considered as empty. Any data previously in that register becomes unusable. Any attempt to use that data will result in an Invalid exception being detected and an [INDEFINITE](#) value possibly being generated.

Although any of the 8 data registers can be tagged as free with this instruction, the only one which can be of any immediate value is the ST(7) register when all registers are in use and another value must be loaded to the FPU. If the data in that ST(7) is still valuable, other instructions should be used to save it before emptying that register.

FDECSTP (Decrement stack pointer)

Syntax: **fdecstp** (no operand)

This instruction simply decrements the value of the [TOP field](#) of the Status Word. If that value was 0, it becomes 7. The contents of the data registers and of the Tag Word are not changed.

From the programmer's point of view, this instruction would cause the content of ST(7) to become the content of the top register and thus become ST(0). The data previously at the top in ST(0) would shift to ST(1), and so on for the other registers.

All data processing instructions for the FPU must use at least the top register as an operand. The main feature of this FDECSTP instruction (and of its counterpart FINCSTP) is to be able to "[rotate the barrel](#)" in order to bring any of the data registers to the top, without changing the content nor their relative order in those registers.

This instruction (or its counterpart FINCSTP) could also be used to bring a free register to the ST(7) position in order to load it with another value. For example, if all registers are full and the value in ST(5) has become useless, the following code could be used.

```
ffree st(5)    ;tags the ST(5) register as empty
fdecstp        ;brings the empty ST(5) register to the ST(6) position
fdecstp        ;brings it now to the ST(7) position
                ;where it could be loaded with some new data
                ;(all the other data has also been shifted two positions)
```

FINCSTP (Increment stack pointer)

Syntax: **fincstp** (no operand)

This instruction simply increments the value of the [TOP field](#) of the Status Word. If that value was 7, it becomes 0. The contents of the data registers and of the Tag Word are not changed.

From the programmer's point of view, this instruction would cause the content of ST(1) to become the content of the top register and thus become ST(0). The data previously in ST(2) would shift to ST(1), and so on for the other registers, the content of ST(0) becoming ST(7).

All data processing instructions for the FPU must use at least the top register as an operand. The main feature of this FINCSTP instruction (and of its counterpart FDECSTP) is to be able to "[rotate the barrel](#)" in order to bring any of the data registers to the top, without changing the content nor their relative order in those registers.

For example, if some operation using the content of both ST(1) and ST(3) is required, the following code could be used if the operation does not affect the location of other registers.

```
fincstp        ;brings ST(1) as ST(0), and ST(3) as ST(2)
fxxx st,st(2) ;perform required operation
fdecstp        ;returns all registers to their original position
```

[RETURN TO](#)
[SIMPLY FPU](#)
[CONTENTS](#)